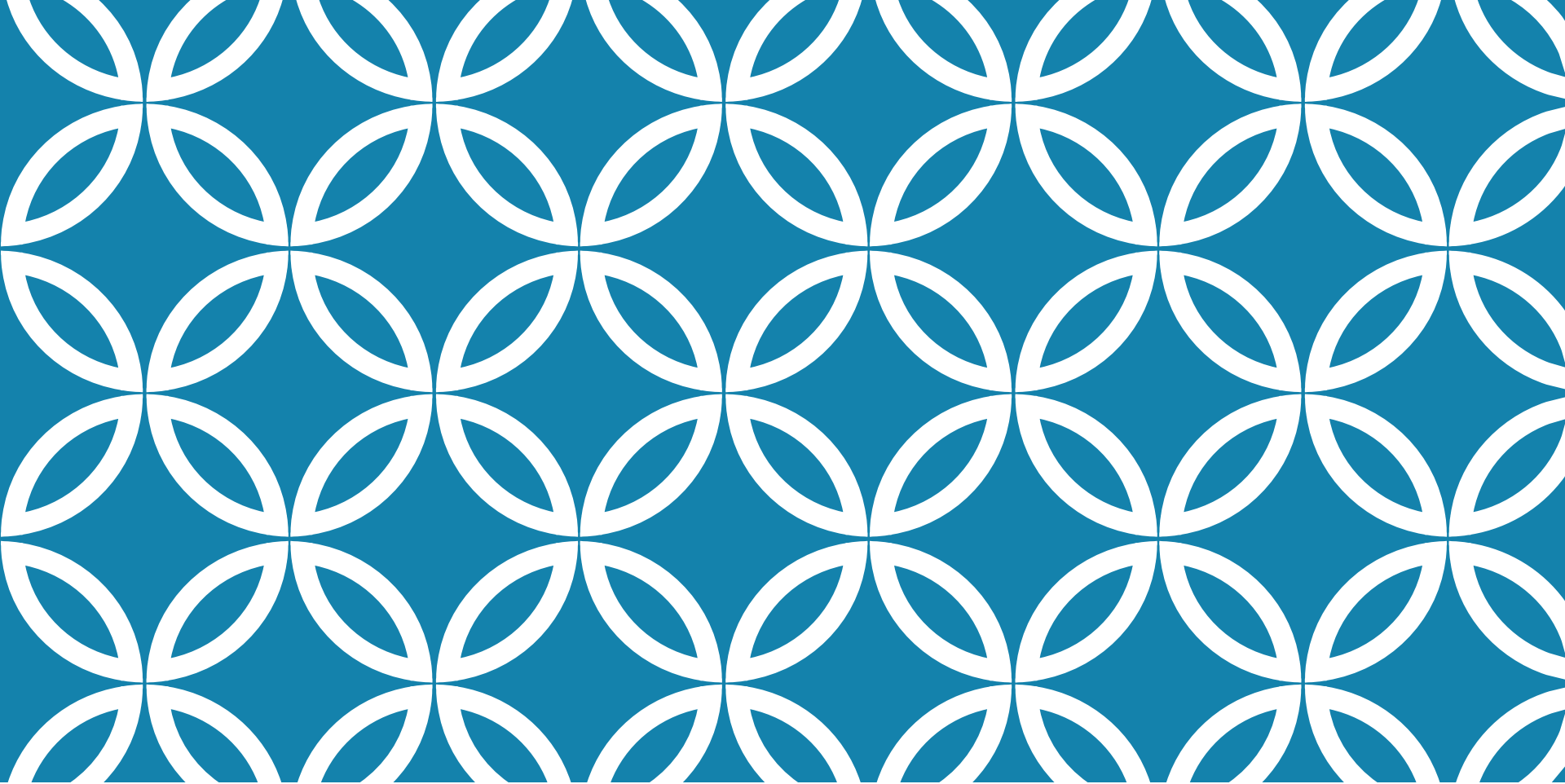


IT SOCIETY

INTRODUCTION

DATABASES

TABLE OF CONTENT

<u>Introduction</u>	7
<u>DQL Commands</u>	8
<u>WHERE</u>	9
<u>DISTINCT</u>	10
<u>ORDER BY</u>	12
<u>ASC</u>	14
<u>DESC</u>	15

TABLE OF CONTENT

(CONTINUES...)

<u>LIMIT</u>	16
<u>OFFSET</u>	19
<u>LIKE</u>	20
<u>%</u>	21
<u>_</u>	23
<u>BETWEEN</u>	25
<u>IS NULL</u>	26

TABLE OF CONTENT

(CONTINUES...)

<u>IS NOT NULL</u>	28
<u>ALIAS</u>	29
<u>Conclusion</u>	30



RELATIONAL DATABASES





QUERYING DATA LANGUAGE

**WHERE,
DISTINCT,
ORDER BY,
LIMIT,
OFFSET,
LIKE,
BETWEEN,
IS NULL/IS NOT
NULL,
AS**

INTRODUCTION

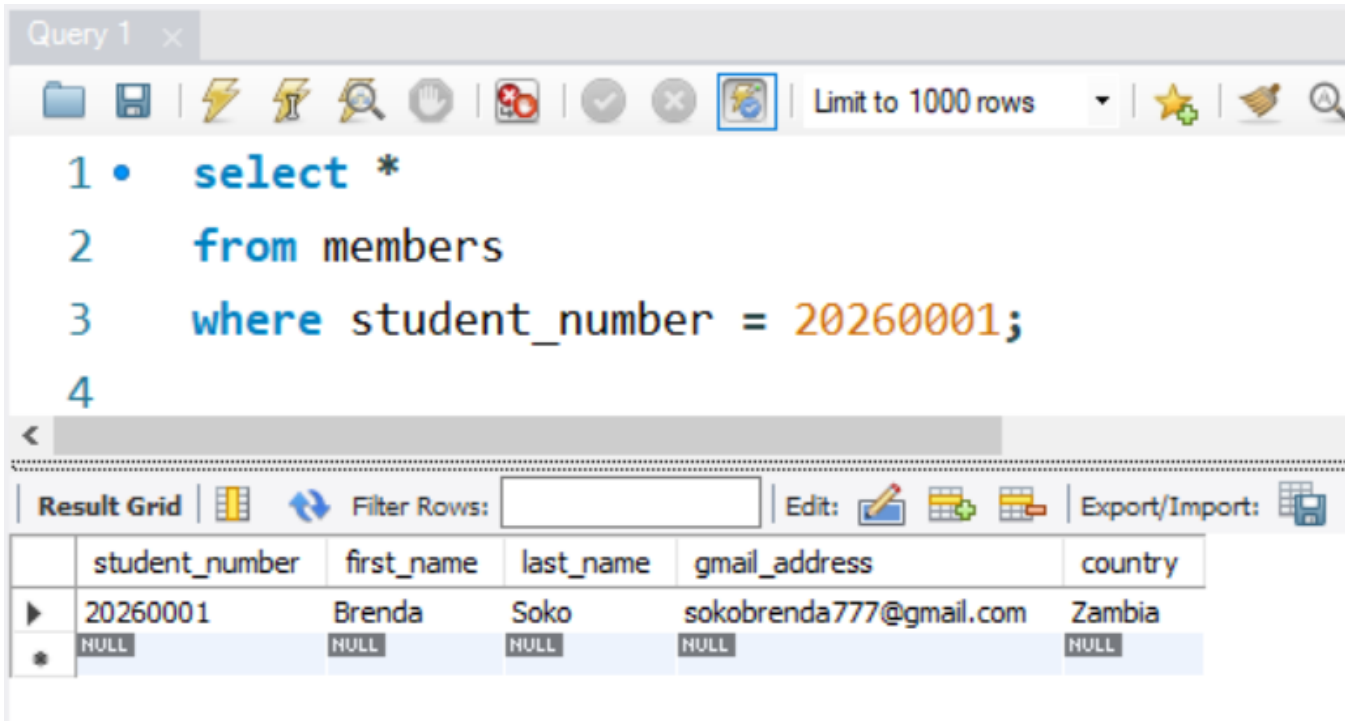
- **Data Query Language (DQL)** is a subset of SQL (Structured Query Language) specifically designed for **querying** and **retrieving** data from databases.
- **MySQL clauses** are the foundational components used to construct database queries.

DQL COMMANDS

COMMAND	DESCRIPTION
WHERE	Filters rows based on a specified condition
DISTINCT	Removes duplicate rows from the result set
ORDER BY	Sorts the result set in ascending (ASC) or descending (DESC) order
LIMIT	Restricts the number of rows returned
OFFSET	Skips a specified number of rows before returning results
LIKE	Matches patterns using wildcards (% for multiple characters, _ for one character)
REGEXP	Matches patterns using regular expressions
BETWEEN	Selects values within a given range (inclusive)
IS NULL	Tests for NULL (missing/unknown) values
IS NOT NULL	Tests for non-NULL values
AS	Creates a temporary alias for a column or table

24. WHERE

- The **WHERE** clause is used to extract only those records that fulfill a specific condition.



The screenshot displays a database query editor window titled "Query 1". The query text is as follows:

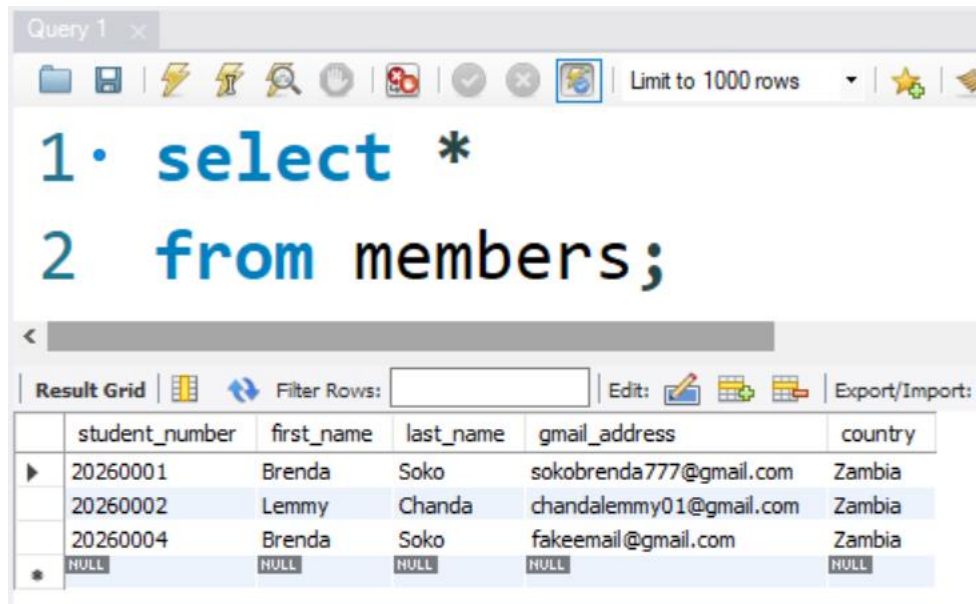
```
1 • select *
2   from members
3   where student_number = 20260001;
4
```

Below the query editor, the "Result Grid" is shown, displaying the results of the query. The grid has five columns: student_number, first_name, last_name, gmail_address, and country. The results are as follows:

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

25. DISTINCT

- The **SELECT DISTINCT** statement is used to retrieve unique values from one or more columns in a table.
- Notice the first_name, and last_name field as duplicate values.



The screenshot shows a database query editor window titled "Query 1". The query text is:

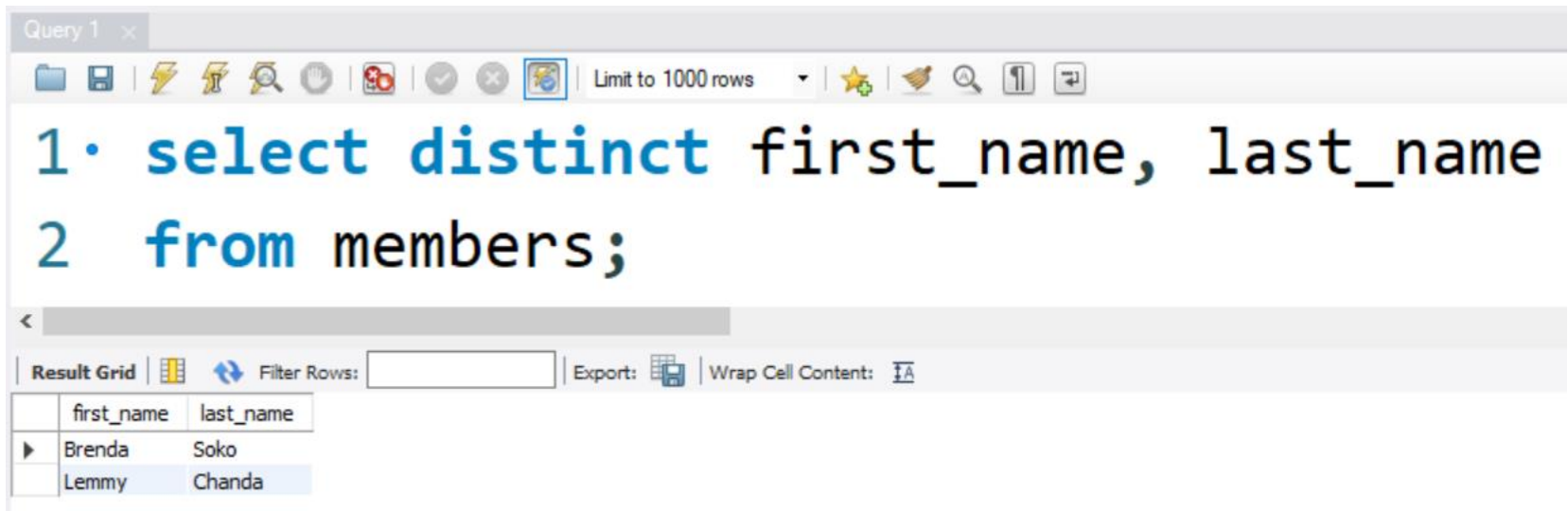
```
1. select *
2. from members;
```

Below the query editor, the "Result Grid" displays the results of the query. The table has the following columns: student_number, first_name, last_name, gmail_address, and country. The results show three rows of data, with the first two rows having duplicate first and last names.

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
	20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

25. DISTINCT (CONTINUES...)

- Now we can select those columns but only extract **unique** records.



The screenshot shows a database query editor window titled "Query 1". The query text is:

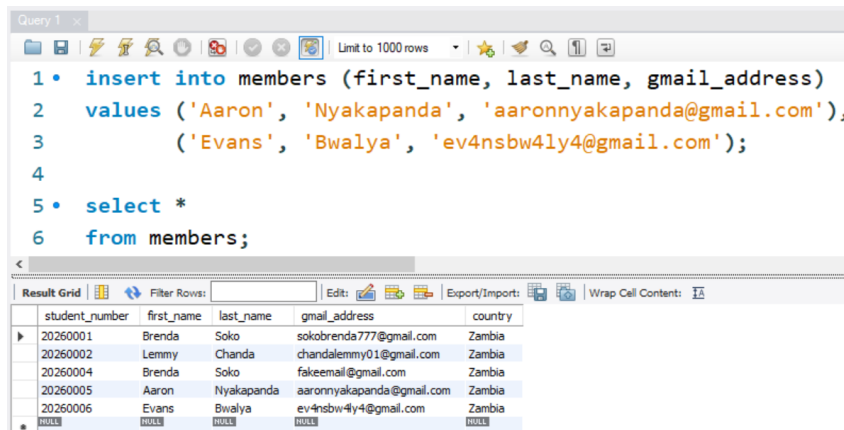
```
1. select distinct first_name, last_name  
2. from members;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has two columns: "first_name" and "last_name". The results are:

first_name	last_name
Brenda	Soko
Lemmy	Chanda

26. ORDER BY

- The **ORDER BY** keyword is used to sort the result-set in **ascending** or **descending** order.
- The **ORDER BY** keyword sorts the records in ascending order (**ASC**) by default.
- Now let's add two more records to test this theory.



The screenshot shows a database query editor with a toolbar at the top. The SQL code is as follows:

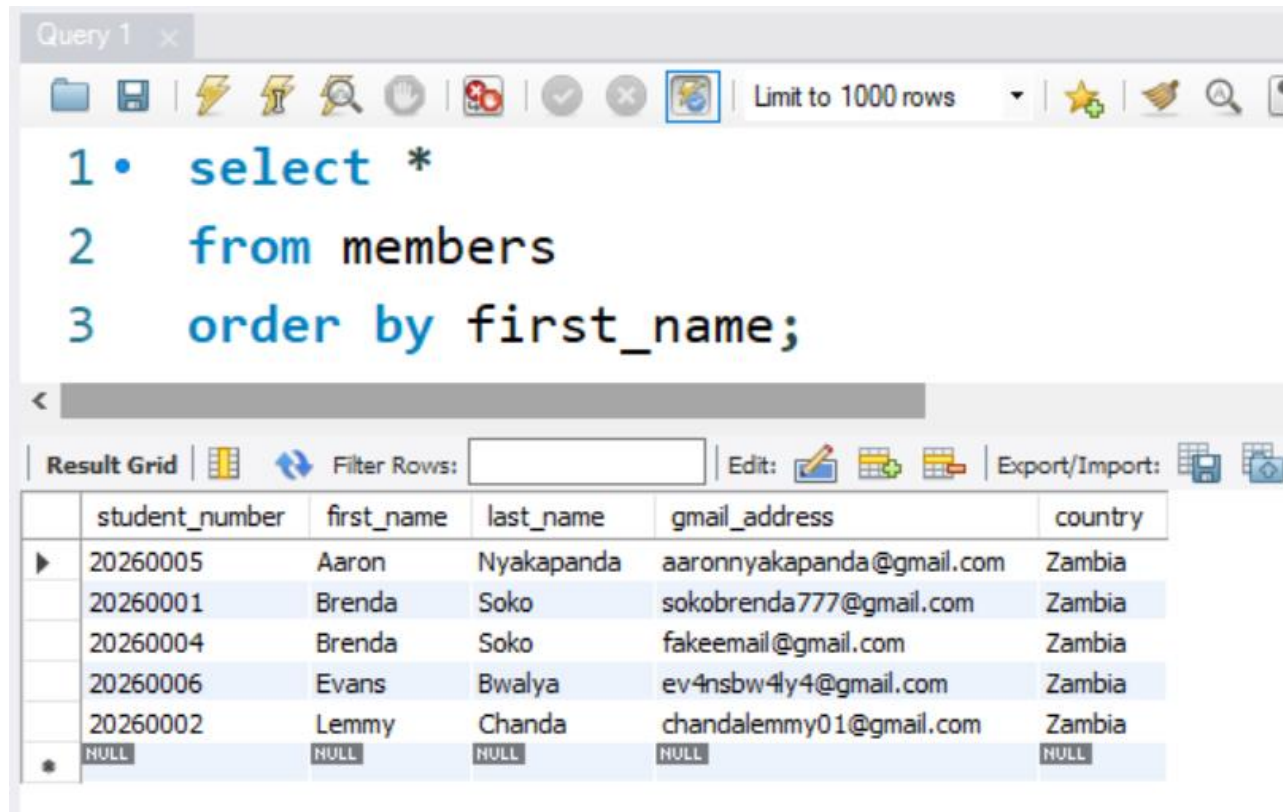
```
1 • insert into members (first_name, last_name, gmail_address)
2 values ('Aaron', 'Nyakapanda', 'aaronnyakapanda@gmail.com'),
3        ('Evans', 'Bwalya', 'ev4nsbw4ly4@gmail.com');
4
5 • select *
6 from members;
```

Below the code is a 'Result Grid' showing the results of the query. The grid has five columns: student_number, first_name, last_name, gmail_address, and country. There are six rows of data, including the two newly inserted records.

student_number	first_name	last_name	gmail_address	country
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
	NULL	NULL	NULL	NULL

26. ORDER BY (CONTINUES...)

- Now we can arrange the records in **ascending** order.



The screenshot shows a database query editor window titled "Query 1". The query is as follows:

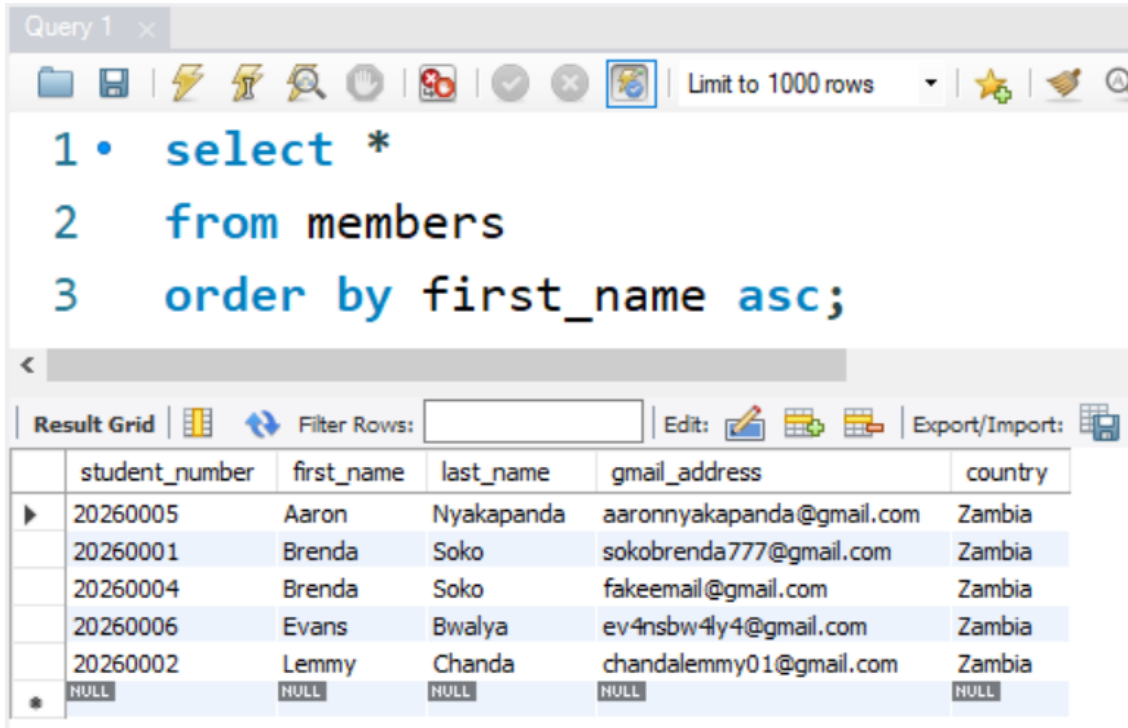
```
1 • select *
2   from members
3   order by first_name;
```

Below the query, the "Result Grid" displays the results of the query. The results are sorted by the first_name column in ascending order. The table has the following columns: student_number, first_name, last_name, gmail_address, and country. The results are as follows:

	student_number	first_name	last_name	gmail_address	country
▶	20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
	20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

26. ORDER BY | ASC

- We can also use the keyword **ASC** to order records in ascending order.



The screenshot shows a database query editor window titled "Query 1". The query is as follows:

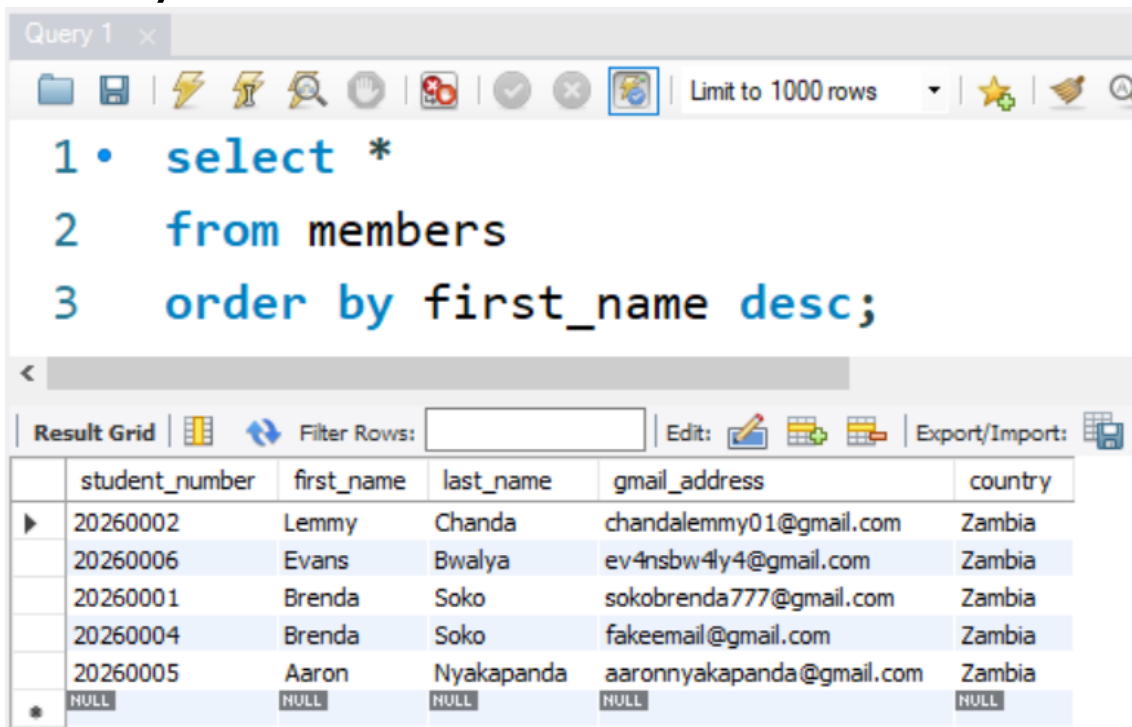
```
1 • select *
2   from members
3   order by first_name asc;
```

Below the query, the "Result Grid" displays the results of the query. The results are ordered by first_name in ascending order. The table has the following columns: student_number, first_name, last_name, gmail_address, and country.

student_number	first_name	last_name	gmail_address	country
20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
NULL	NULL	NULL	NULL	NULL

26. ORDER BY | DESC

- We can also order records in **descending** order by using the keyword **DESC**.



The screenshot shows a database query editor window titled "Query 1". The query text is:

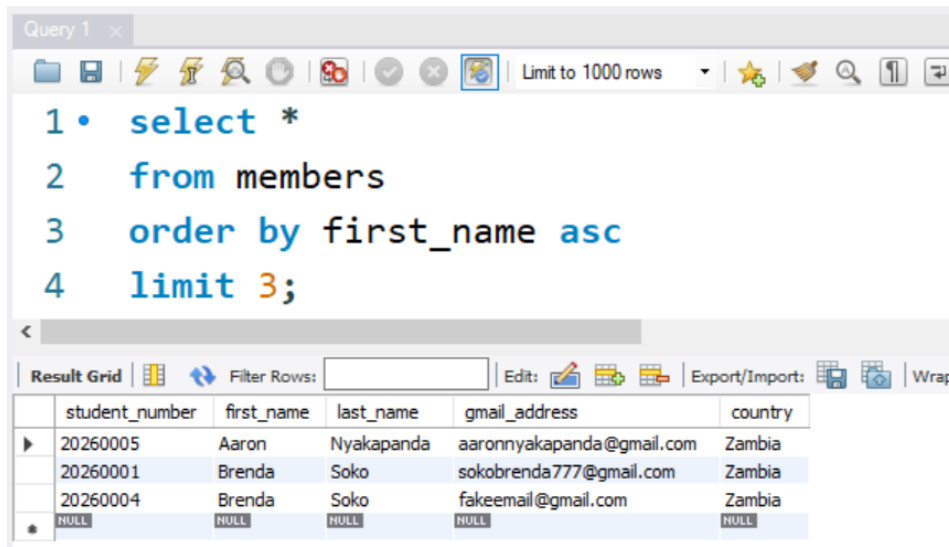
```
1 • select *
2   from members
3   order by first_name desc;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The results are ordered by first_name in descending order. The table has the following columns: student_number, first_name, last_name, gmail_address, and country.

student_number	first_name	last_name	gmail_address	country
20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
NULL	NULL	NULL	NULL	NULL

27. LIMIT

- The **LIMIT** clause is used to limit the number of records to return.
- Unlike the previous query where we returned all the records, here we can limit the records.



The screenshot shows a database query editor window titled "Query 1". The SQL query entered is:

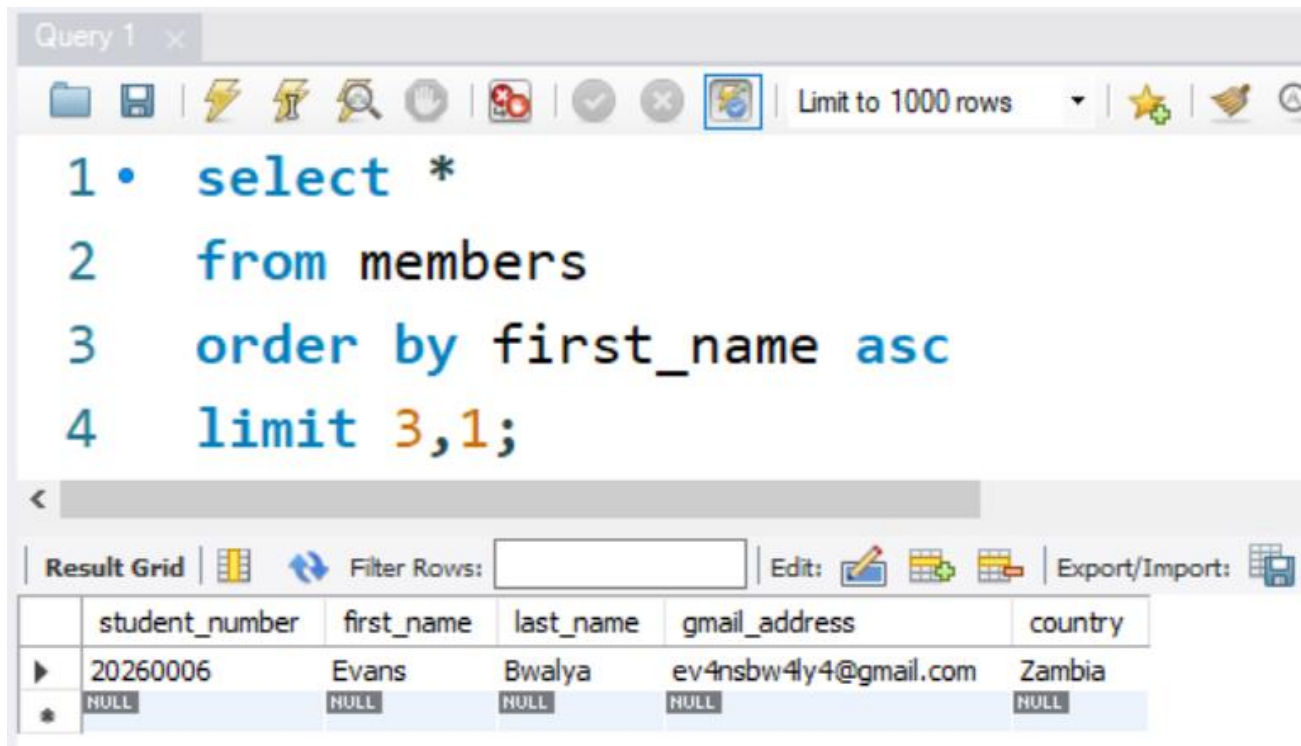
```
1 • select *
2 from members
3 order by first_name asc
4 limit 3;
```

The results are displayed in a table with the following columns: student_number, first_name, last_name, gmail_address, and country. The results are limited to 3 rows.

student_number	first_name	last_name	gmail_address	country
20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260004	Brenda	Soko	fakeemail@gmail.com	Zambia

27. LIMIT (CONTINUES...)

- We can ignore some records and limit some.



The screenshot shows a database query editor window titled "Query 1". The query is as follows:

```
1 • select *
2   from members
3   order by first_name asc
4   limit 3,1;
```

The results are displayed in a table with the following columns: student_number, first_name, last_name, gmail_address, and country. The first row shows a student with student_number 20260006, first_name Evans, last_name Bwalya, gmail_address ev4nsbw4ly4@gmail.com, and country Zambia. The second row is highlighted with a blue background and shows NULL values for all columns.

	student_number	first_name	last_name	gmail_address	country
▶	20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

27. LIMIT (CONTINUES...)

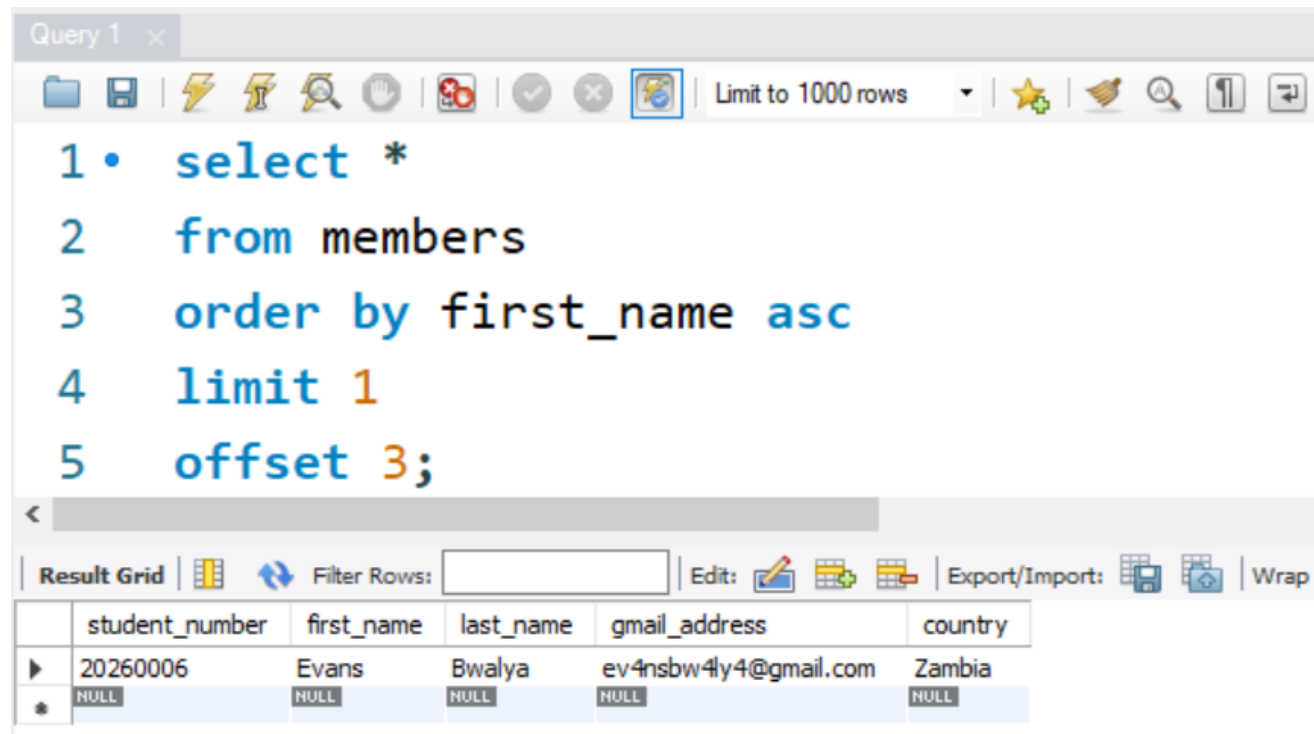
- **NOTE:** limit 3,1 means 3 is the **OFFSET** and 1 is the **LIMIT**.
- But LIMIT can not be used to ignore the middle part?

	student_number	first_name	last_name	gmail_address	country
▶	20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
	20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

- For example, you can't ignore the record in red.

28. OFFSET

- The **OFFSET** clause in MySQL is used to **skip** a specified number of rows before starting to return rows from a query.



The screenshot shows a MySQL query editor window titled "Query 1". The query text is as follows:

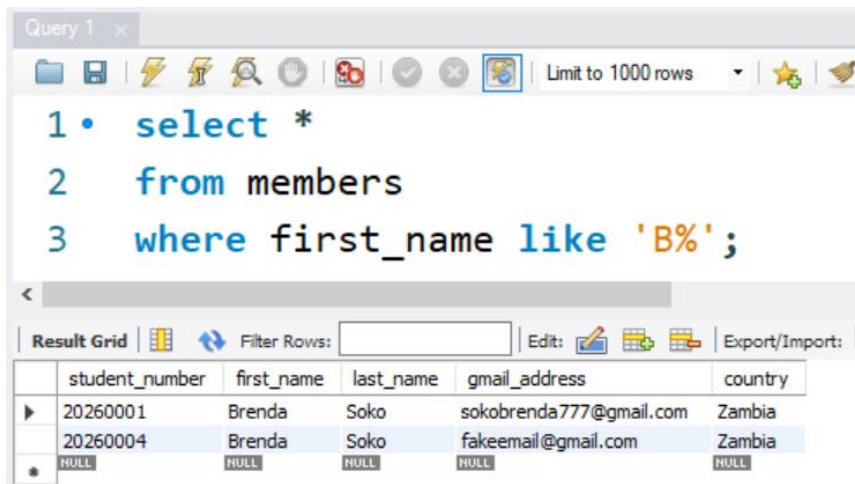
```
1 • select *
2   from members
3   order by first_name asc
4   limit 1
5   offset 3;
```

Below the query editor, the "Result Grid" is displayed. It shows a single row of data from the "members" table, ordered by first_name. The row contains the following values:

	student_number	first_name	last_name	gmail_address	country
▶	20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

29.1 LIKE | MULTIPLE PARTEN SEARCH %

- The **LIKE** operator is used in a **WHERE** clause to **search for a specified pattern** within a column's text data.
- The % (percent sign) is a **wildcard character** that represents **multiple characters**.
- We can search the **starting part** of a record.



The screenshot shows a database query editor window titled "Query 1". The query text is:

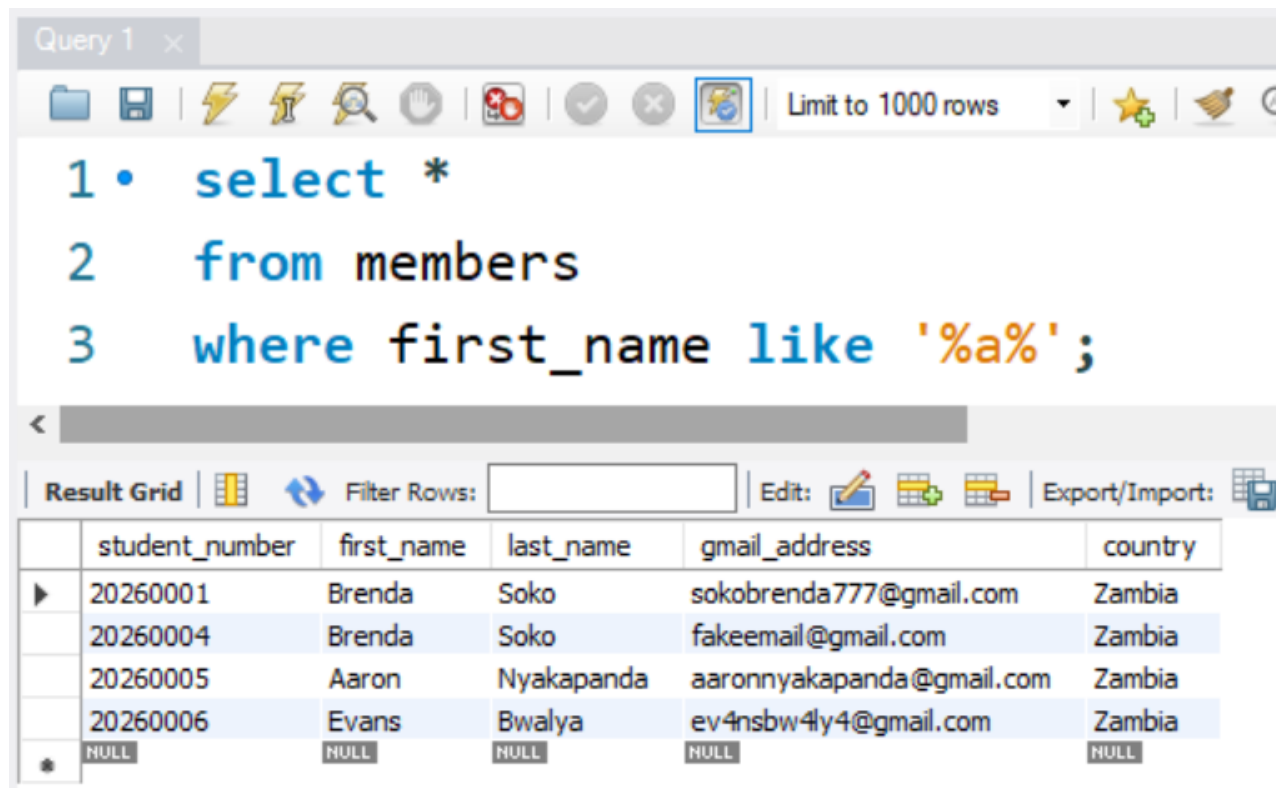
```
1 • select *
2   from members
3   where first_name like 'B%';
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns for student_number, first_name, last_name, gmail_address, and country. The results are as follows:

student_number	first_name	last_name	gmail_address	country
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
NULL	NULL	NULL	NULL	NULL

29.1 LIKE | MULTIPLE PARTEN SEARCH % (CONTINUES...)

- We can search the **middle part** of a record.



The screenshot shows a database query editor window titled "Query 1". The query text is as follows:

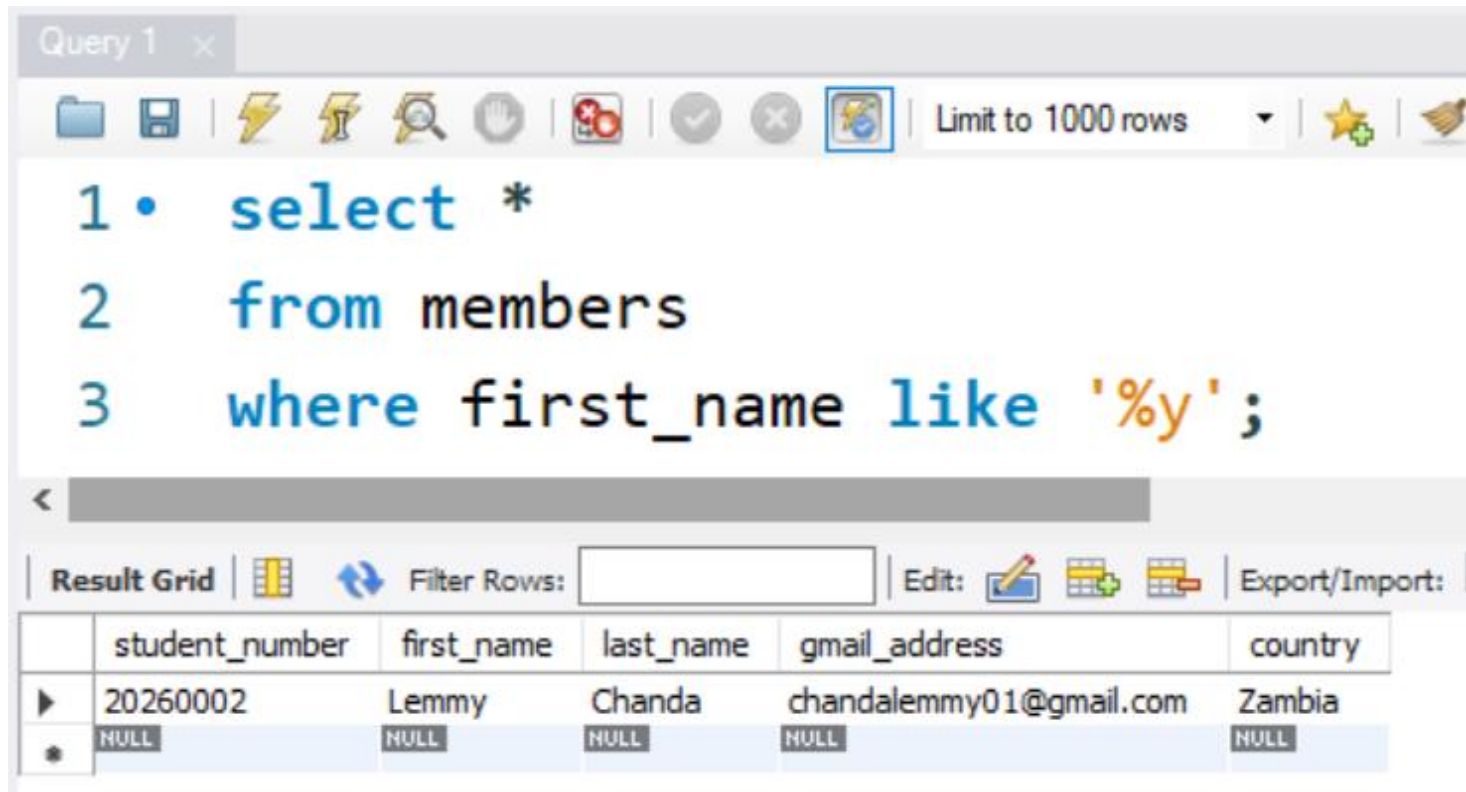
```
1 • select *
2   from members
3   where first_name like '%a%';
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns for student_number, first_name, last_name, gmail_address, and country. The results are as follows:

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
	20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
	20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

29.1 LIKE | MULTIPLE PARTEN SEARCH % (CONTINUES...)

- We can search the **last part** of a record.



The screenshot shows a database query editor window titled "Query 1". The query is as follows:

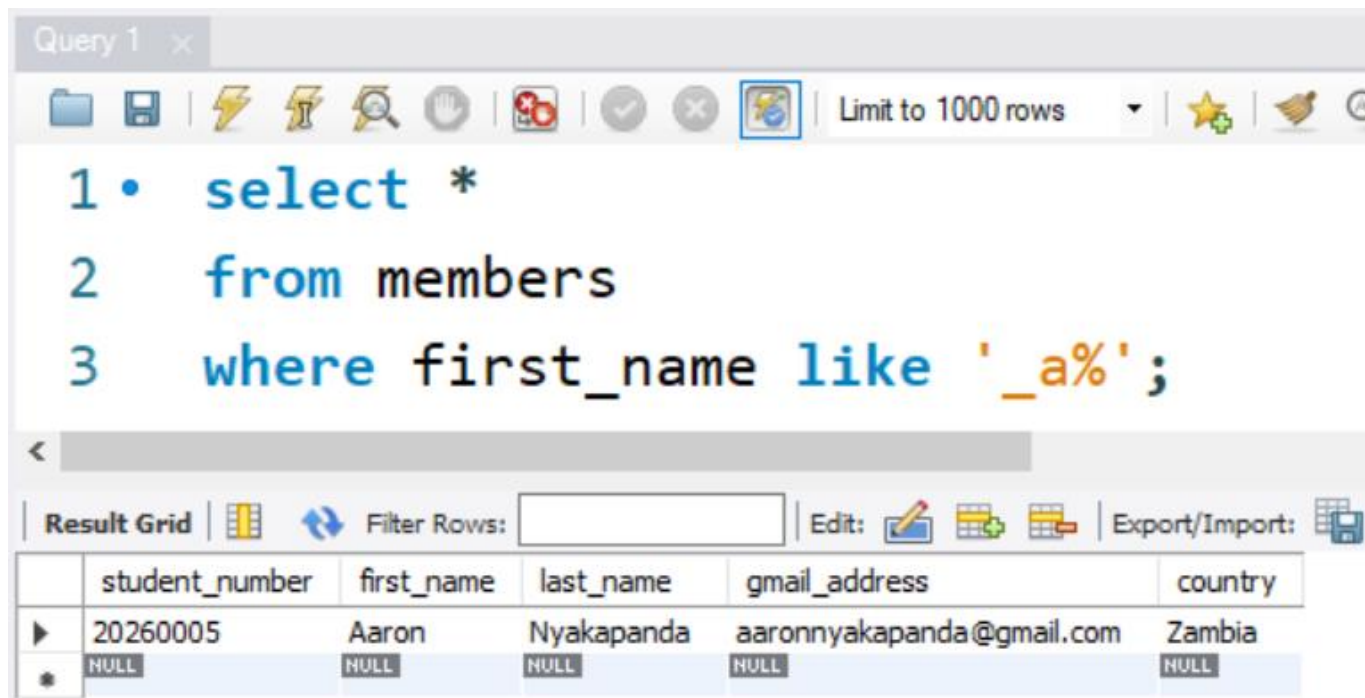
```
1 • select *
2   from members
3   where first_name like '%y';
```

Below the query editor, there is a "Result Grid" section. It includes a "Filter Rows:" input field and buttons for "Edit:", "Export/Import:", and "Filter Rows:". The results are displayed in a table with the following columns: student_number, first_name, last_name, gmail_address, and country.

	student_number	first_name	last_name	gmail_address	country
▶	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

29.2 LIKE | SINGLE PARTEN SEARCH _

- The underscore _ is a **wildcard character** that represents exactly **one single character**.



The screenshot shows a database query editor interface. At the top, there's a tab labeled 'Query 1'. Below it is a toolbar with various icons for file operations, execution, and settings. The main area contains a SQL query:

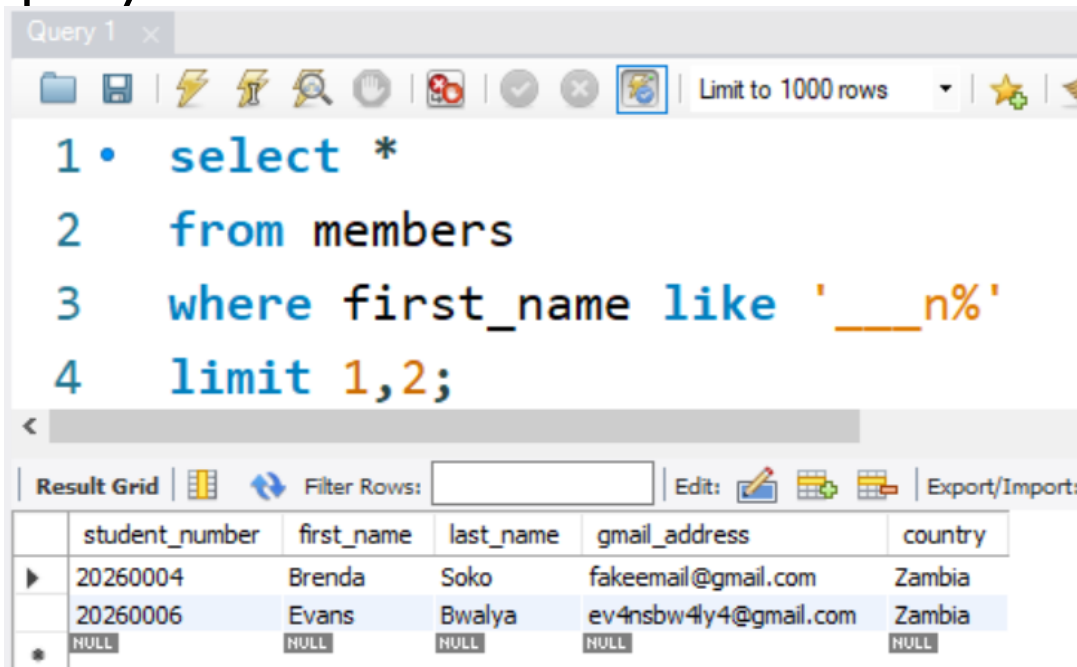
```
1 • select *
2   from members
3   where first_name like '_a%';
```

Below the query editor is a 'Result Grid' section. It includes a 'Filter Rows' input field and buttons for 'Edit', 'Export/Import', and other grid actions. The results are displayed in a table with the following columns: student_number, first_name, last_name, gmail_address, and country.

	student_number	first_name	last_name	gmail_address	country
▶	20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
•	NULL	NULL	NULL	NULL	NULL

29.2 LIKE | SINGLE PARTEN SEARCH _ (CONTINUES...)

- We can also use multiple _ for example Brenda and Evans happens to have an n on position number 4, so the query will be –



The screenshot shows a database query editor window titled "Query 1". The query is as follows:

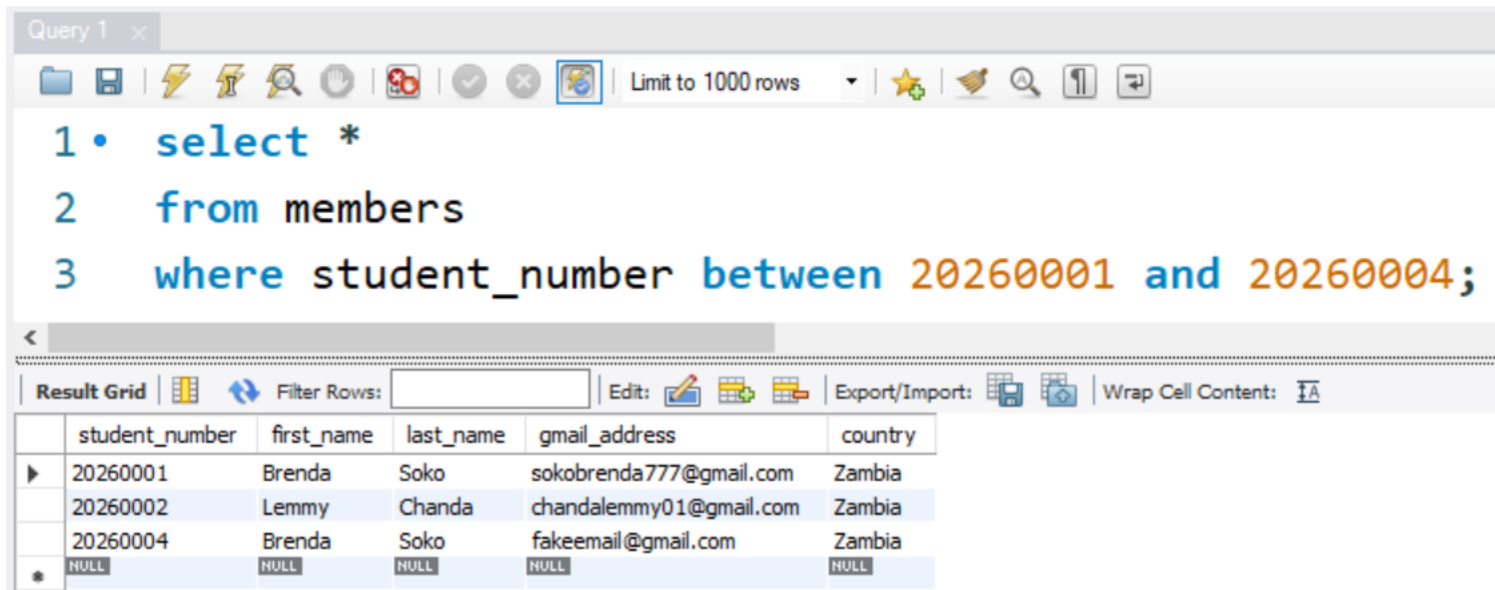
```
1 • select *
2   from members
3   where first_name like '____n%'
4   limit 1,2;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns for student_number, first_name, last_name, gmail_address, and country. The results are as follows:

	student_number	first_name	last_name	gmail_address	country
▶	20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
	20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

30. BETWEEN

- The **BETWEEN** operator is used in the **WHERE** clause to select values within a **specified range**.
- For example, our student_number range from 20260001 – 20260006 but we can just decide the range we want.



The screenshot shows a database query editor interface. The query editor displays the following SQL query:

```
1 • select *  
2   from members  
3   where student_number between 20260001 and 20260004;
```

Below the query editor, the results are displayed in a table grid. The table has the following columns: student_number, first_name, last_name, gmail_address, and country. The results show three rows of data, plus a summary row with NULL values.

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
	20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

31.1 IS NULL

- The **IS NULL** operator is used to test for **empty values** (NULL values).
- For demonstration, let's create a record with some null values.

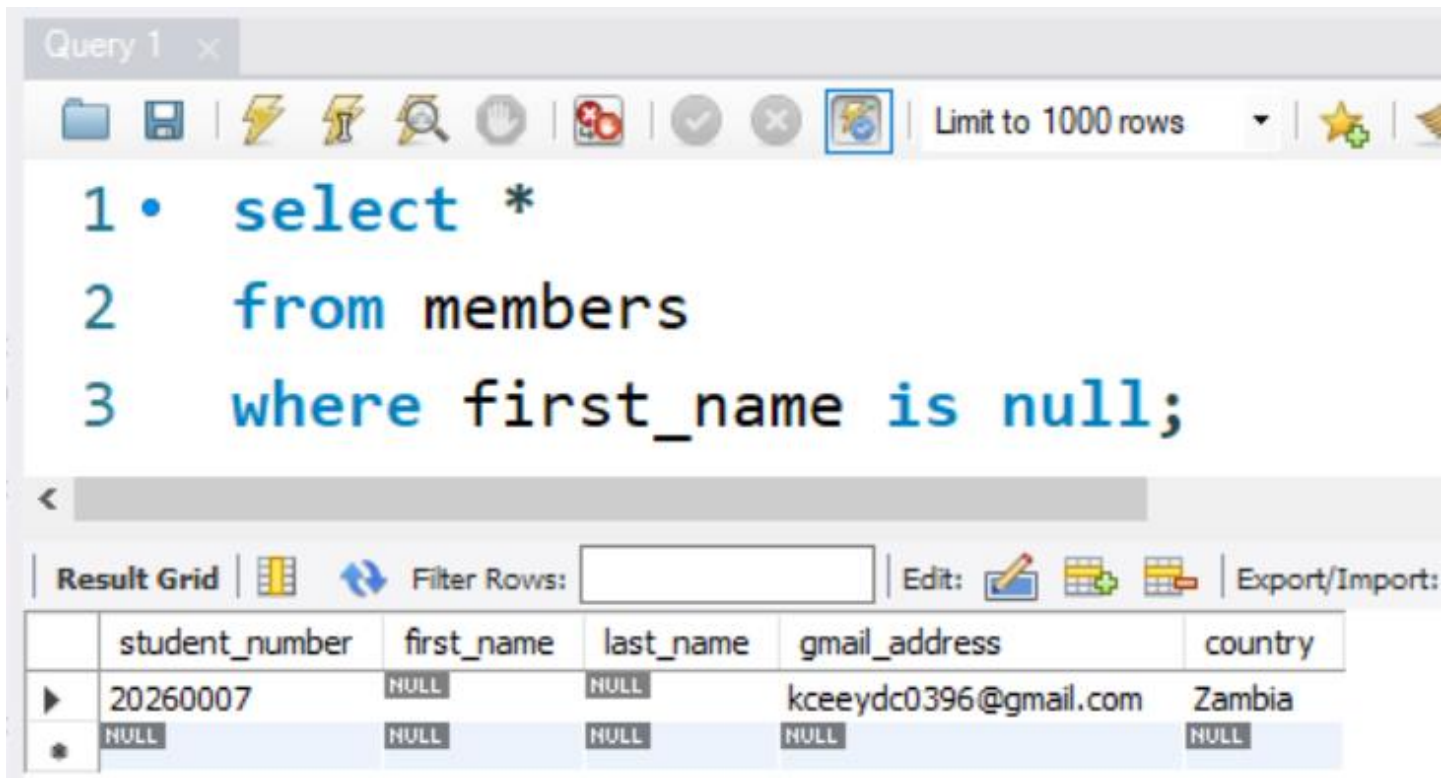
The screenshot shows a database query tool interface. The top pane displays an SQL query with five lines: an insert statement to add a new member with a specific email address, followed by a select statement to retrieve all records from the 'members' table. The bottom pane shows the 'Result Grid' with a table containing seven rows of member data. The last row, corresponding to the newly inserted record, shows NULL values for first_name, last_name, and country, while the email_address is 'kceeydc0396@gmail.com'.

```
1 • insert into members (gmail_address)
2   values ('kceeydc0396@gmail.com');
3
4 • select *
5   from members;
```

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
	20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
	20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
	20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
	20260007	NULL	NULL	kceeydc0396@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

31.1 IS NULL (CONTINUES...)

- Now we can search for the NULL values.



The screenshot shows a database query tool interface. At the top, there's a tab labeled "Query 1". Below it is a toolbar with various icons for file operations, execution, and filtering. The main area displays a SQL query:

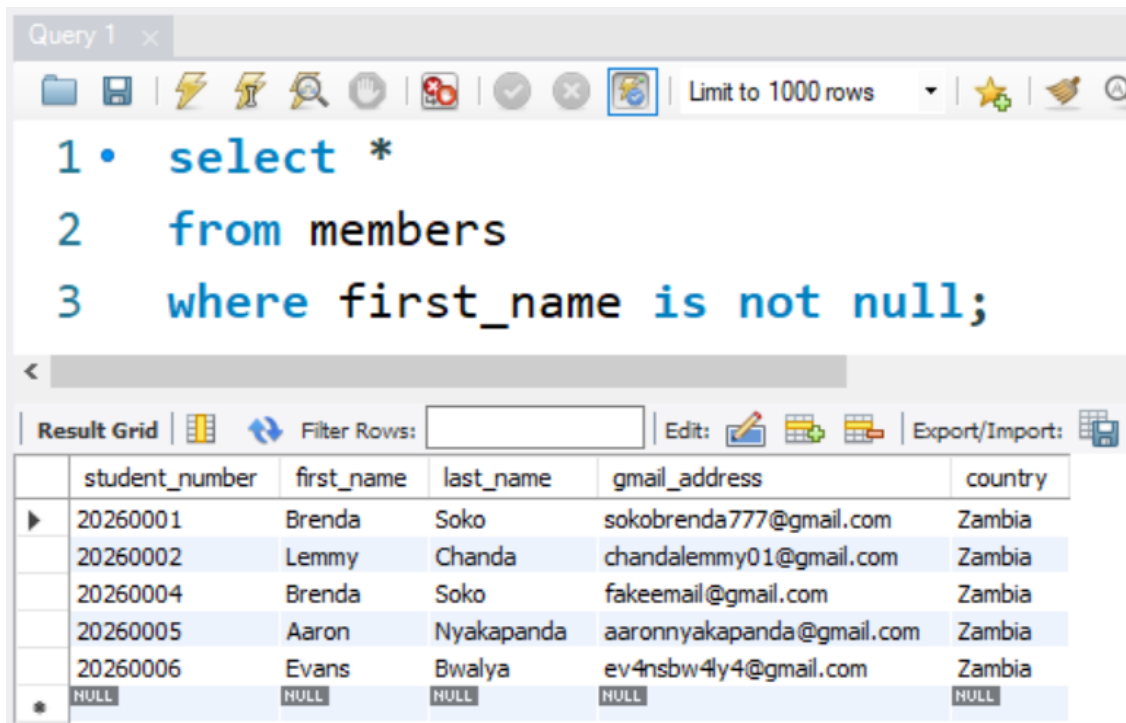
```
1 • select *
2   from members
3   where first_name is null;
```

Below the query editor, there's a "Result Grid" section. It includes a "Filter Rows:" input field and buttons for "Edit:", "Export/Import:", and "Filter Rows:". The results are displayed in a table with the following columns: student_number, first_name, last_name, gmail_address, and country.

	student_number	first_name	last_name	gmail_address	country
▶	20260007	NULL	NULL	kceeydc0396@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

31.2 IS NOT NULL

- The **IS NOT NULL** operator is used to test for non-empty values (NOT NULL values).



The screenshot shows a database query editor window titled "Query 1". The query text is:

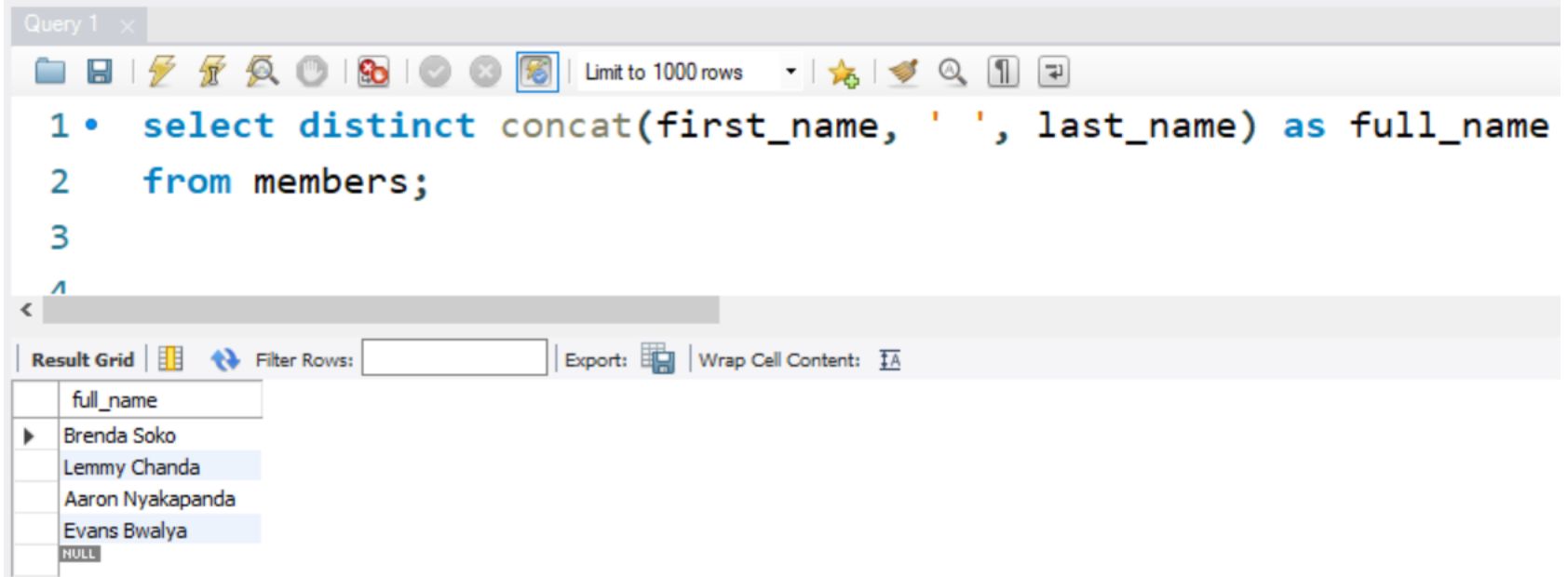
```
1 • select *
2   from members
3   where first_name is not null;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns for student_number, first_name, last_name, gmail_address, and country. The results are as follows:

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
	20260004	Brenda	Soko	fakeemail@gmail.com	Zambia
	20260005	Aaron	Nyakapanda	aaronnyakapanda@gmail.com	Zambia
	20260006	Evans	Bwalya	ev4nsbw4ly4@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

32. ALIAS

- The **AS** keyword in MySQL is used to assign an **alias** to a table or column, making it easier to reference or improving readability.



The screenshot shows a MySQL query editor window titled "Query 1". The query is as follows:

```
1 • select distinct concat(first_name, ' ', last_name) as full_name
2   from members;
3
4
```

Below the query editor, the "Result Grid" is displayed, showing the output of the query. The first column is labeled "full_name". The results are:

full_name
Brenda Soko
Lemmy Chanda
Aaron Nyakapanda
Evans Bwalya
NULL

CONCLUSION

- At this point, we now have an understanding about some **DQL** commands in **MySQL**.
- In this handout, our focus was to understand the basic use of **WHERE, DISTINCT, ORDER BY, LIMIT, OFFSET, LIKE, BETWEEN, IS NULL, IS NOT NULL** and **AS**.



UP NEXT... OPERATORS